

# Data Structures Homework 1: Answer Sheet

Student ID: 114703020 Name: 張又琦

## 1. Shortest counterexample for the faulty growth strategy

### 1.1 Shortest operation sequence

The initial capacity is 2. Fill in at most 5 `enqueue()`/`dequeue()` operations:

```
CircularQueue myQ(2);
myQ.enqueue(1); myQ.enqueue(2);
myQ.dequeue();
myQ.enqueue(3); myQ.enqueue(4);
```

### 1.2 Step-by-step state

| Operation   | front_ | rear_ | size_ | Logical order | Actual array layout |
|-------------|--------|-------|-------|---------------|---------------------|
| .enqueue(1) | 0      | 1     | 1     | (1,_)         | (1,_)               |
| .enqueue(2) | 0      | 0     | 2     | (1,2)         | (1,2)               |
| .dequeue()  | 1      | 0     | 1     | (2,_)         | (1,2)               |
| .enqueue(3) | 1      | 1     | 2     | (2,3)         | (3,2)               |
| .enqueue(4) | 0      | 3     | 3     | (3, 2, 4, _)  | (3, 2, 4, _)        |

### 1.3 Correct versus faulty result

- Elements removed in order by the correct implementation: 2->3->4
- Elements removed in order by the faulty implementation: 3->2->4

### 1.4 Which invariant is broken

**Ans:**the First-In First-Out principle.

## 2. Operation counts and growth trend

**Ans:**Big-O notation is not required in this section.

### 2.1 CircularQueue

- When timing starts, what are `size()` and `capacity()`? **Ans:**The `size()` and `capacity()` are both  $n$ , which represents the elements stored in `sizes[]`. During the whole operation, it has been  $2^9$ ,  $2^{12}$ , and  $2^{15}$ .

- Each round runs `dequeue()` first and then `enqueue()`. How many existing elements does each of the two operations move? **Ans:**The existing elements are never moved. The `front_` and `rear_` are moved, but not the elements themselves.
- Why does no capacity growth occur during the timed phase? **Ans:**Because there is no need. `dequeue()` leaves one space unused, and `enqueue()` uses that space. Minus 1 and plus 1, so it never runs out of spaces.

## 2.2 NaiveShiftQueue

- With `n` elements in the queue, roughly how many elements does each `dequeue()` shift forward? **Ans:**`n-1`
- After `n` consecutive rounds, roughly how many element moves are performed in total? **Ans:**`n-1 + n-1 + n-1 + ... = n(n-1)`

## 2.3 Growth when the input size increases

| Implementation               | When <code>n</code> grows 8 times, the number of basic operations grows roughly how many times? | Reasoning                                                                                                                                                                                            |
|------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CircularQueue</code>   | 8                                                                                               | every <code>enqueue()</code> and <code>dequeue()</code> only requires one basic operation                                                                                                            |
| <code>NaiveShiftQueue</code> | 64                                                                                              | <code>enqueue()</code> only requires one basic operation, but <code>dequeue()</code> traverses the whole queue, so the whole <code>measure()</code> requires roughly the square of the queue's size. |

## 3. Experimental results and discussion

### 3.1 Median running times and growth ratios

| <code>n</code> | <code>CircularQueue</code> median (ms) | Ratio to previous size | <code>NaiveShiftQueue</code> median (ms) | Ratio to previous size |
|----------------|----------------------------------------|------------------------|------------------------------------------|------------------------|
| $2^9$          | 0.008                                  | n/a                    | 0.647                                    | n/a                    |
| $2^{12}$       | 0.066                                  | 8.25                   | 19.248                                   | 29.750                 |
| $2^{15}$       | 0.214                                  | 3.242                  | 864.558                                  | 44.917                 |

### 3.2 Comparing operation counts with measured times

**Ans:**The ratio of `CircularQueue`:  $2^{12}/2^9$  is very close to 8, but that of `CircularQueue`:  $2^{15}/2^{12}$  is less than expected. That might be because the CPU spent more resources on the operation when `n` got bigger and therefore increased the efficiency. The ratio of both `NaiveShiftQueue`:  $2^{12}/2^9$  and  $2^{15}/2^{12}$  are less than expected. That might have resulted from the fixed costs like initializing the variables, which make the calculated ratio not showing obviously when `n` is not very big.